

Postal Delivery Operations System

Complete system guide: the backend, the admin panel and the Postman field app

What it does, how it is built, how every part works together, and how to run and test it.

Snapshot of the codebase at D:\Projects\POSTMANROUTE

Generated 20 September 2026

Contents

1. What this system is	2
The life of a parcel	2
2. Roles and access	2
3. Data model	3
4. The backend	5
Every request goes through the same gates	5
API reference	5
Delivery import and assignment	8
Delivery status rules	9
Other backend behaviour	9
5. Route optimization	10
6. The admin panel	11
7. The Postman app	12
Signing in and sessions	12
The three tabs	12
Offline and sync	13
8. Configuration	13
9. Running and testing	14
10. Known limitations	14

1. What this system is

The Postal Delivery Operations System runs a post office's delivery round from the moment a parcel list arrives to the moment a postman marks a parcel delivered. It has three parts that share one backend and one database:

- **Backend** - an Express API (TypeScript) on PostgreSQL with the PostGIS spatial extension. It owns all data, rules, authentication, geocoding, route planning and reporting.
- **Admin panel** - a React web app for post office staff: import parcel lists, assign parcels to beats and postmen, resolve exceptions, manage staff and vehicles, watch the round on a map, and export reports.
- **Postman app** - an Expo / React Native app (iOS, Android, and a web build) for postmen in the field: see today's parcels in delivery order, follow an optimized road route on a map, navigate to each address, and record outcomes, even offline.

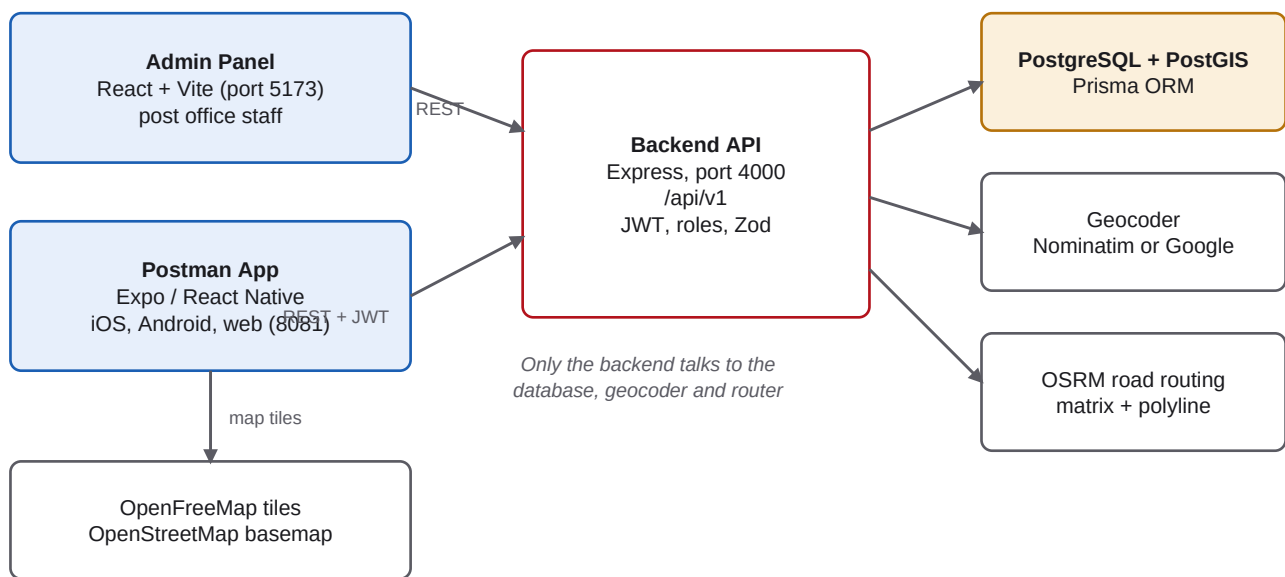


Figure 1. Both clients call the same REST API. Only the backend touches the database, the geocoder and the routing engine.

The pilot deployment data is for Bhandup West Post Office, Mumbai (code MUM-BHW-400078), but nothing in the schema or the routing logic is tied to that office: every record is scoped by post office.

The life of a parcel

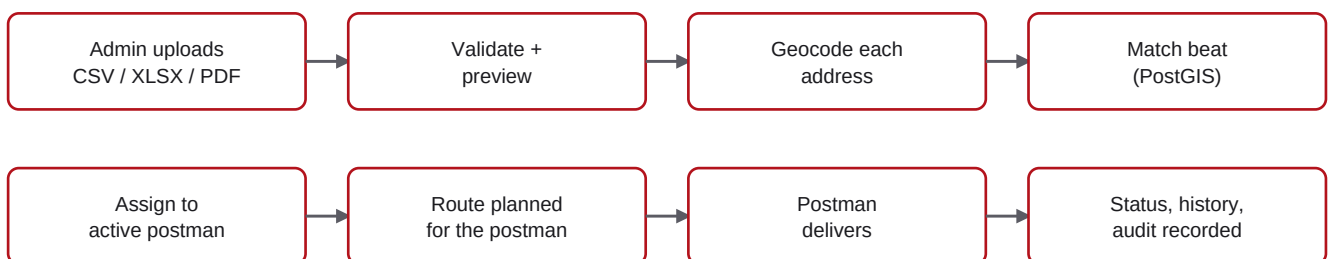


Figure 2. End-to-end flow. Anything that cannot be resolved automatically becomes an assignment exception for an admin to fix.

2. Roles and access

Three roles exist. Every request carries a signed JWT access token; the role inside it decides what the caller may do, and the post office inside it decides which data they may see.

Role	Who	Can do
SUPER_ADMIN	Head office / platform owner, no single home office	Everything an admin can do across all post offices (may pass ?postOfficeId=). Only role that can create post offices and admin accounts (create, deactivate, activate).
ADMIN	Post office staff, attached to one post office	Imports, assignments, beats, postmen, vehicles, deliveries, reports, audit log, data quality, optimization requests - only for their own post office. Creating something in another post office is refused; reading it returns 404.
POSTMAN	Field worker, linked to one Postman record	Only the self-service /me/* endpoints, plus reading and updating status of deliveries assigned to themselves. Another postman's delivery looks like it does not exist (404).

Identity for postmen is never taken from the request body: the backend looks up which Postman the logged-in user is linked to (User.postmanId) on every call, so a postman cannot act as someone else by sending a different id.

3. Data model

The database is PostgreSQL with PostGIS. Prisma defines the schema; spatial columns (beat boundaries, live locations) use PostGIS geometry / geography types and are queried with raw SQL.

Model	Purpose and key fields
PostOffice	The tenant. Code, name, address, pincode, latitude / longitude (used as the default route start / depot), active flag.
User	A login. Email, argon2 password hash, role, status (ACTIVE / INACTIVE / SUSPENDED), post office, optional postmanId link, lastLoginAt.
RefreshToken	Hashed (SHA-256) refresh tokens with expiry and revocation time; one-time-use rotation.
Beat	A delivery territory. Number, name, PostGIS polygon boundary, centre point, status, free-form metadata. Unique per post office.
Postman	Field worker. Employee id, contact details, status (ACTIVE / ON_LEAVE / INACTIVE), assigned beat, live currentLocation (PostGIS point), lastActiveAt.
PostmanBeatAssignment	History of who covered which beat (start / end date, isActive). At most one active row per beat and per postman.
Vehicle	Asset id, model, battery capacity (Wh), battery %, status (AVAILABLE, ASSIGNED, IN_USE, CHARGING, MAINTENANCE, INACTIVE), assigned postman. There is no payload / capacity field.
VehicleBatteryLog	Time series of battery percentage per vehicle.
Recipient / Address	Who and where. Address holds lines, area, city, state, pincode, latitude, longitude, PostGIS location, geocoding status (PENDING, SUCCESS, FAILED, MANUAL), source and confidence.
Delivery	The parcel job. Tracking id (unique), recipient, address, parcelType, parcelCount, priority (LOW, NORMAL, HIGH, URGENT), urgency text, serviceTimeMinutes, status, post office, beat, assigned postman. There is no weight field.
DeliveryStatusHistory	Every status change: from, to, reason, who, when.
DeliveryAttempt	One row per outcome recorded by a postman: delivery, postman, outcome status, notes, time.
DeliveryAssignmentHistory	Who / which beat a delivery was assigned to, why, by whom, and whether it was a manual override.
AssignmentException	A parcel that needs a human: reason (GEOCODING_FAILED, NO_BEAT_MATCH, MULTIPLE_BEAT_MATCH, INVALID_ADDRESS, INVALID_COORDINATES, NO_POSTMAN_ASSIGNED, INACTIVE_POSTMAN, OUTSIDE_POST_OFFICE), details, resolvedAt, lastAction.
DeliveryImport / DeliveryImportRow	An uploaded file and its rows: counts (valid, invalid, duplicate, missing, geocoding failed, assignment failed, successful), column mapping, status (UPLOADED, PROCESSING, PREVIEW_READY, CONFIRMED, CANCELLED, FAILED), per-row raw data, normalized data and errors.
PostmanLocationHistory	GPS pings from the app: latitude, longitude, battery %, isMock flag, time.
OptimizationRequest / OptimizationResult	Where planned routes live. The request records the postman, delivery ids, trigger and algorithm; the result holds the whole route as JSON (stops, legs, ETAs, polyline, cost).

Model	Purpose and key fields
Route / RouteStop / RouteEvent	Present in the schema but not used by any code path today (routes are stored in the optimization tables).
Notification	Message, type, severity (INFO, WARNING, CRITICAL), read flag, optional user.
AuditLog	Who did what to which entity: action, old / new value, reason, IP address, time.

4. The backend

The backend is an Express 4 application (TypeScript, Node) using Prisma 5, Zod for validation, pino for logging, helmet, cors, compression and express-rate-limit. All business endpoints live under `/api/v1`.

Every request goes through the same gates



Figure 3. Request pipeline. Errors are returned as JSON: `{ "error": { "message", "details" } }`. Prisma unique-constraint errors become 409, missing records 404, anything unexpected 500 with a generic message.

Authentication and sessions

- Login (POST `/auth/login`) checks the email, that the account is ACTIVE and the argon2id password hash; it is rate limited to 20 attempts per 15 minutes. It returns a short-lived access token (JWT, 15 minutes by default) and a refresh token (7 days).
- Refresh tokens are random, stored only as SHA-256 hashes, and are **rotated on every use**: the old one is revoked and a new one issued. Logout revokes the presented token.
- The JWT carries user id, role, post office and email. Login and logout are written to the audit log.
- Interactive API docs are served at `/api/docs` (Swagger UI from openapi.yaml). GET `/health` returns `{"status": "ok"}`.

Multi-post-office isolation

Listing endpoints filter by the caller's post office. Single-record endpoints re-check the record's post office against the caller (so guessing an id from another office returns 404), and create endpoints refuse a post office id that is not the caller's. Postmen additionally can only reach their own deliveries.

API reference

All paths are relative to `/api/v1`. "Admin" means ADMIN or SUPER_ADMIN. "Any" means any logged-in user (results are still scoped to the caller's post office).

Authentication

Method	Path	Who	What it does
POST	<code>/auth/login</code>	Public	Email + password (min 8 chars) -> access token, refresh token, user (incl. postmanId). Rate limited.
POST	<code>/auth/refresh</code>	Public	Rotate a refresh token -> new access + refresh token.
POST	<code>/auth/logout</code>	Any	Revoke the refresh token; audit logged.
GET	<code>/auth/me</code>	Any	The current user's profile.

Postman self-service (POSTMAN only)

Method	Path	Who	What it does
GET	/me/profile	Postman	Own postman record, assigned beat, last known location.
GET	/me/stats	Postman	Today's total, completed, failed and remaining counts plus completion rate.
GET	/me/deliveries	Postman	Own deliveries with recipient, address, beat. Filters: status, page, pageSize (max 100).
GET	/me/route	Postman	Current optimized route (see section 6). Query: startLat + startLng, refresh=true, algorithm. Returns { route: null } when nothing is left.
POST	/me/route/reoptimize	Postman	Force a full re-plan. Body: trigger, optional start {latitude, longitude}, optional algorithm. Audit logged.
POST	/me/location	Postman	GPS ping: latitude, longitude, optional battery %, isMock. Stores history and updates the live location.

Deliveries

Method	Path	Who	What it does
GET	/deliveries	Any	Search and filter: q (tracking id, name, phone, pincode, address), status, priority, beatId, postmanId, dateFrom, dateTo, page, pageSize.
GET	/deliveries/:id	Any	Full detail with status history, assignment history and exceptions (postmen: own only).
POST	/deliveries/:id/status	Admin, Postman	Change status. Validated against the allowed-transition table; writes history, attempt, audit; may publish an event.
POST	/deliveries/:id/reassign	Admin	Manual override of beat and / or postman with a required reason; assignment history + audit.
POST	/deliveries/bulk	Admin	ASSIGN_BEAT, ASSIGN_POSTMAN or SET_PRIORITY for many deliveries at once (checked to belong to the admin's office).

Imports, geocoding and exceptions

Method	Path	Who	What it does
GET	/imports	Any	Import history.
GET	/imports/:id	Any	One import with counts and status.
GET	/imports/:id/rows	Any	The preview rows with per-row status and errors.
POST	/imports/upload	Admin	Upload a CSV, XLSX or PDF (multipart). XLSX with several sheets asks for ?sheet=. Returns the preview.
PUT	/imports/:id/mapping	Admin	Override the detected column mapping and re-validate.
POST	/imports/:id/confirm	Admin	Geocode valid rows, create deliveries, match beats, assign postmen.
POST	/imports/:id/cancel	Admin	Cancel an unconfirmed import.
POST	/geocoding/retry/:addressId	Admin	Re-run geocoding for an address; on success re-runs beat assignment.
POST	/geocoding/manual/:addressId	Admin	Set coordinates by hand (status MANUAL).
GET	/assignments/exceptions	Any	Open (or all) assignment exceptions.
POST	/assignments/exceptions/:id/resolve	Admin	Mark an exception resolved with the action taken.
POST	/assignments/exceptions/:id/ignore	Admin	Ignore an exception.

Beats, postmen, vehicles, post offices, users

Method	Path	Who	What it does
GET / POST	/beats	Any / Admin	List (with GeoJSON boundary) / create a beat with a polygon boundary.
GET / PUT	/beats/:id	Any / Admin	Read / update a beat.
POST	/beats/:id/activate deactivate	Admin	Change beat status.
POST	/beats/:id/assign-postman	Admin	Assign a postman (or none) to the beat. One active postman per beat.
GET	/beats/:id/workload	Any	Delivery counts grouped by status for the beat.
GET / POST	/postmen	Any / Admin	List with filters / create a postman.
GET / PUT / DELETE	/postmen/:id	Any / Admin	Read one / update / delete.
POST	/postmen/:id/activate deactivate	Admin	Change postman status.
POST	/postmen/:id/assign-beat	Admin	Assign a beat; the same one-active-assignment rule applies.
GET / POST / PUT	/vehicles	Any / Admin	List / create / update vehicles.
POST	/vehicles/:id/battery-log	Admin	Record a battery reading.
GET / POST	/post-offices	Any / Super	List / create post offices (create is SUPER_ADMIN only).
GET	/post-offices/:id	Any	One post office.
GET / POST	/users	Super	List / create admin accounts (SUPER_ADMIN only).
POST	/users/:id/activate deactivate	Super	Enable or disable an account.

Maps, dashboard, reports, data quality, audit, notifications, research, optimization

Method	Path	Who	What it does
GET	/maps/beats deliveries postmen post-offices	Any	GeoJSON feature collections for the admin map.
GET	/dashboard/summary	Any	Cards (total, assigned, unassigned, delivered, pending, failed, rescheduled, active postmen and beats), open exceptions, 5 recent imports, per-beat workload.
GET	/reports/postmen	Any	Per-postman assigned / delivered / failed / rescheduled and success rate. ?format=csv downloads.
GET	/reports/beats	Any	Per-beat totals, completed, pending, failed. ?format=csv downloads.
GET	/reports/daily	Any	One day's totals (?date=).
GET	/data-quality/summary	Any	Row-level import quality: valid, invalid, duplicate, missing, geocoding failures, no beat match, no postman.
GET	/data-quality/errors	Any	Up to 1000 problem rows; ?format=csv downloads.
POST	/data-quality/retry/:importRowId	Admin	Retry a failed import row.
GET	/audit-logs	Admin	Paginated audit trail.
GET	/notifications	Any	The caller's notifications.
POST	/notifications/:id/read /read-all	Any	Mark one or all notifications read.
POST / GET	/optimization/requests	Admin / Any	Admin-triggered route planning for chosen deliveries; read a request and its result.
GET	/research/density clusters ripley benchmarks	Any	Return stored research results only; empty with available:false until a research engine publishes some.

Delivery import and assignment

This is how a parcel list becomes assigned work:

- **Upload.** Admins upload a CSV, XLSX or text-based PDF (limit MAX_UPLOAD_SIZE_MB, default 25 MB). The file type is checked by extension and MIME type and stored under a random name. A scanned PDF is rejected as FAILED ("OCR is required but not available"); an XLSX with several sheets asks which sheet to use.
- **Column mapping.** Headers are matched to system fields using known aliases (for example awb, consignment_no, mobile, pin, landmark). The admin can override any mapping.
- **Validation.** Required: recipient name, address, city, state, phone, pincode. Phones must be 10-digit Indian mobile numbers starting 6-9; pincodes 6 digits. A missing tracking id is generated. Rows are marked VALID, INVALID, MISSING_DATA or DUPLICATE.
- **Preview and confirm.** Nothing is created until an admin reviews the preview and confirms. Cancelling discards it.
- **Geocoding.** Each valid row is geocoded (Nominatim by default in code, or Google). Nominatim tries the full address, then area + city + pincode (confidence 0.4), then city + pincode (0.25), and caches results in the database. Failure creates a GEOCODING_FAILED exception.
- **Beat matching.** A PostGIS ST_Contains test finds which active beat polygon contains the point - never nearest-centre distance. No match, several matches, or no active postman on the beat each create an exception and leave the parcel SORTED.
- **Assignment.** One match plus an ACTIVE postman assigns the parcel (status ASSIGNED) and writes assignment history. Admins clear exceptions by retrying geocoding, correcting the address, assigning manually or changing the beat; every override is audited.

Delivery status rules

The server owns the state machine: a change is accepted only if it appears in this table. DELIVERED, RETURNED and CANCELLED are final. The apps mirror the table to decide which buttons to show, but the backend always re-checks.

From	May move to
RECEIVED	SORTED, CANCELLED
SORTED	ASSIGNED, CANCELLED
ASSIGNED	OUT_FOR_DELIVERY, CANCELLED, RESCHEDULED
OUT_FOR_DELIVERY	DELIVERED, RECIPIENT_UNAVAILABLE, REJECTED, WRONG_ADDRESS, ADDRESS_NOT_FOUND, FAILED
RECIPIENT_UNAVAILABLE / WRONG_ADDRESS / ADDRESS_NOT_FOUND / FAILED	RESCHEDULED, RETURNED
REJECTED	RETURNED
RESCHEDULED	OUT_FOR_DELIVERY, CANCELLED
DELIVERED, RETURNED, CANCELLED	(nothing - final)

When a change is accepted the server: updates the delivery, writes a status-history row, writes a delivery-attempt row (if a postman is assigned), writes an audit-log row, and - for RECIPIENT_UNAVAILABLE, REJECTED, WRONG_ADDRESS and CANCELLED - publishes a delivery event. A listener now consumes those events and re-plans the assigned postman's remaining route (it does so without ever failing the status update).

Other backend behaviour

- **Audit log.** Logins, logouts, imports, assignments and overrides, status changes, user, postman, beat and vehicle changes, and route generation are recorded with user, entity, old / new value, reason and IP.
- **Reports and dashboard.** Computed live from the tables (no separate reporting store); CSV export for postman, beat and data-quality reports.
- **Notifications.** The table, service and REST endpoints exist, but nothing in the backend creates notifications yet, so the list is empty in practice. Server-driven push to the mobile app is also not built (no push-token endpoint).
- **Research endpoints.** Density, DBSCAN clusters, Ripley's K and algorithm benchmarks return only stored results and are empty until a research engine writes them. They never return fabricated data.

5. Route optimization

Each postman gets a real, road-based route for the deliveries they still have to do. It is planned by the backend; the Postman app only displays it.

Which deliveries are routed

Only deliveries assigned to the postman with status ASSIGNED, OUT_FOR_DELIVERY or RESCHEDULED. DELIVERED, RETURNED, CANCELLED and failed-attempt deliveries (RECIPIENT_UNAVAILABLE, REJECTED, WRONG_ADDRESS, ADDRESS_NOT_FOUND, FAILED) are excluded. Stops with missing or invalid coordinates are listed as "unroutable" with a reason instead of failing the route.

Where the route starts

In order: the GPS position sent by the app, else the postman's latest location ping (if under 12 hours old), else the post office's coordinates. Nothing is hard-coded.

Travel times and the road line

A road travel-time and distance matrix comes from an OSRM routing engine (OSRM_BASE_URL). Every answer is cached per pair of points, so re-planning after a delivery needs no new matrix request. The road polyline for the final order is fetched separately (long routes are stitched from several requests). If OSRM is not configured, times out, or fails, the route falls back to straight-line estimates (great-circle distance x 1.3 at 25 km/h) and says so: the response is flagged `routing.mode: ESTIMATED` with a warning, and the app draws a dashed guide line and shows "Road route unavailable".

The two strategies

	NN_2OPT (default)	DBSCAN_NN_2OPT
Idea	Treat the round as one tour.	Group stops into neighbourhoods first, finish one before the next.
Step 1	Nearest Neighbor: repeatedly go to the stop with the lowest weighted cost from where you are.	DBSCAN over the road-time matrix (default: neighbours within 300 s, minimum 2 stops per cluster). Stops in no cluster become single-stop clusters.
Step 2	2-opt: reverse any segment whose reversal lowers the full route cost; repeat until nothing improves.	Visit clusters nearest-first; Nearest Neighbor inside each cluster.
Step 3	-	2-opt inside each cluster, then 2-opt over the cluster order, repeated until stable. Clusters always stay contiguous.
Trade-off	Typically the shorter of the two.	Compact, predictable sub-rounds; can be slightly longer (about 4% on the demo data with a 90 s radius).

The cost function

Both strategies minimise the same cost, computed over the whole route:

```
cost = travel + loadPenalty + priorityPenalty

travel          = sum of leg travel times (seconds, from the road matrix)
loadPenalty     = loadWeight (0.35) x sum( leg time x parcels still on board / total parcels )
priorityPenalty = priorityWeight (0.25) x sum( priorityFactor x arrival time )
                priorityFactor: LOW 0, NORMAL 0, HIGH 1, URGENT 3
                arrival time includes earlier stops' dwell time (default 3 min)
```

Because there is no weight field, **parcel count stands in for the load carried**: driving a leg with a fuller bag costs more, so bigger drops are pulled earlier and long legs are pushed to when the bag is light. Urgent and high-priority parcels are penalised for waiting. With both weights at zero the cost collapses to plain shortest travel time. The two weights are my defaults, not calibrated against real data; they can be overridden per request.

Keeping the route fresh

- The finished route is stored (in the optimization tables) and returned unchanged while the postman's set of active deliveries is unchanged.

- If stops are only **removed** (delivered, failed, reassigned) the order is kept: finished stops are dropped and legs, ETAs and the road line are refreshed. The remaining stops never reshuffle just because one was completed.
- If a delivery is **added**, on a manual Recalculate, or after a failure event, a full optimization runs again.
- Work is serialised per postman; if a refresh fails, the last stored route is returned marked stale rather than an error.
- 2-opt is bounded by a pass limit (500) and a time budget (1.5 s) so a large route can never block the API.
- Switching strategy is explicit (request parameter, env default, or the app's Route method switch) and then sticks for that postman's route.

6. The admin panel

A React 18 + Vite single-page app (React Router, TanStack Query, react-hook-form + Zod, MapLibre with a drawing tool). It calls the backend through a same-origin `/api` proxy and refreshes its tokens automatically. Pages after login:

Page	What it does
Dashboard	Summary cards (total, assigned, unassigned, delivered, pending, failed, rescheduled, active postmen and beats), open assignment exceptions, five most recent imports, and beat workload with high workload flagged rather than judged.
Deliveries / Delivery detail	Search and filter parcels; open one to see its status history, assignment history and exceptions; reassign or change status; bulk assign beat or postman and set priority.
Assignment Exceptions	The work queue for parcels that could not be assigned automatically: retry geocoding, correct the address and reassign, assign manually, or ignore.
Beats	Manage territories: create / edit / activate / deactivate; assign the covering postman; view workload.
Postmen / Postman detail	Add and edit postmen, activate / deactivate, assign a beat; personal information and historical performance.
Map	All beats as polygons, delivery locations, postmen and post offices; draw a New Territory polygon directly on the map.
Imports / Import detail	Upload CSV, XLSX or PDF; review the preview with per-row problems; adjust column mapping; confirm or cancel; import history.
Data Quality	Counts of invalid, duplicate and missing-data rows, geocoding failures, no beat matched and assignment failures; export problem rows as CSV.
Reports	Postman performance, beat report and daily report, with CSV export.
Audit Log	Paginated trail of who did what and when.
Admin Accounts	SUPER_ADMIN only: create, activate and deactivate admin accounts.
Vehicles, Notifications, Research Analytics	Pages exist in the source for fleet and battery tracking, notifications, and placeholders for density, DBSCAN, Ripley's K and benchmark results. They are not routed or linked in the current app (the navigation lists Dashboard, Deliveries, Assignment Exceptions, Beats, Postmen, Map, Imports, Data Quality, Reports, Audit Log, Admin Accounts).

7. The Postman app

An Expo (SDK 57) / React Native app with TypeScript, React Navigation, TanStack Query, Zustand and axios. It runs on iOS and Android and also as a web build. It is for POSTMAN accounts only, and it never implements routing itself: it displays what the backend plans.

Signing in and sessions

- Login accepts only a POSTMAN-role account that is linked to a Postman profile; anything else is refused with a clear message and nothing is saved.
- Tokens are kept in secure storage on the device (never plain storage). On launch the saved session is restored.
- A request that fails with 401 triggers one shared token refresh; if that fails the user is logged out. Logging out revokes the refresh token and clears cached data.
- Server error messages (for example "Invalid delivery status transition") are shown to the postman, not a generic failure.

The three tabs

Deliveries

- Header: date, name, today's Total / Completed / Remaining / Failed, and a one-line route summary (stops to go, distance, time; "estimated distances" when there is no road routing).
- Filter chips: All, Pending, Completed, Failed, Rescheduled. Pull-to-refresh reloads deliveries, route and stats.
- Cards are ordered by the route: numbered stops in delivery order, then any unrouted parcels, then finished ones. The next stop is marked NEXT.
- **Collapsed card:** stop number (or a tick / cross when finished), recipient, priority, status, distance and time from the previous stop, address.
- **Tap to expand:** address, parcels (count and type), priority, estimated arrival, tracking id, beat, cluster number (DBSCAN routes), and the action buttons. Only fields the Delivery model really has are shown - there is no weight.
- **Navigate** opens turn-by-turn directions to the delivery's coordinates (Apple Maps on iOS, Google Maps on Android, a Google Maps link elsewhere); if coordinates are unusable it searches the written address.
- **Start Delivery** (ASSIGNED / RESCHEDULED to OUT_FOR_DELIVERY) and **Mark Delivered** (OUT_FOR_DELIVERY to DELIVERED, after a confirmation). A parcel that is still ASSIGNED never offers Mark Delivered.
- "Full details" opens a detail screen with history, a Call Recipient button, and every other allowed outcome (unavailable, wrong address, failed, reschedule, return, cancel) with an optional reason.

Map

- Road-following route line with direction arrows (solid = real road geometry, dashed = straight-line guide when routing is unavailable).
- Numbered markers in delivery order; the next stop is blue and larger; done stops show a tick, failed stops a cross; the selected stop has a ring. The route start is marked S; the device GPS position is a blue dot.
- A top banner answers "what is next, how far, how do I get there": stop number, recipient, address, distance, time and ETA, with Navigate and Recalculate buttons.
- Tapping a marker selects it and opens a bottom sheet with the same Navigate / Start / Mark Delivered buttons as the card. Tapping a card focuses the map on it; the two tabs share one selection.
- A Route method switch chooses NN + 2-opt or DBSCAN + NN + 2-opt; the summary card names the method and the number of clusters. Location permission is optional (the start comes from the server).
- After a delivery the marker, the card and the route update together; the remaining order is preserved.

Account

- Today's completed, failed and pending counts and completion rate.
- Profile (name, phone, email, employee id, assigned beat, status, last known location - managed by the post office admin), Settings (location permission, notification permission, about), an ID Card (postman identity card), and Log Out.

Offline and sync

The app has one offline system, used by every status change:

- Connectivity is tracked with NetInfo. When offline, a status change goes into a persistent queue on the device instead of failing, and the postman sees "Saved offline". The card and marker immediately show the status the queue will apply, tagged "Waiting to sync".
- If the app thinks it is online but the request fails at the network level, the change is queued rather than lost.
- When connectivity returns, the queue is replayed **oldest first**. A real network failure stops the replay and keeps the rest queued.
- If the server rejects a queued change (someone already moved the parcel on), the app never overwrites the server: it drops the stale change, re-reads the delivery, and shows a banner such as "Couldn't apply 'Out for Delivery' for Kavita Rao: it is now Cancelled on the server" with a Dismiss button.
- The last deliveries list and route are cached so the app can show something on a cold, offline launch.
- GPS: while a route is active the device pings its position roughly every 30 seconds (every 3 minutes when idle); pings are best-effort and never shown as errors.

8. Configuration

Backend settings (environment variables, see backend/.env.example). Only DATABASE_URL and the two JWT secrets are required.

Variable	Default	Meaning
DATABASE_URL	-	PostgreSQL connection string (database needs the PostGIS extension).
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	-	Signing secrets. Change from the example values.
JWT_ACCESS_EXPIRES_IN / JWT_REFRESH_EXPIRES_IN	15m / 7d	Token lifetimes.
PORT, NODE_ENV, CORS_ORIGIN	4000, development, http://localhost:5173	Server port and the single browser origin allowed by CORS.
UPLOAD_DIR, MAX_UPLOAD_SIZE_MB	./uploads, 25	Import file storage and size limit.
GEOCODING_PROVIDER, NOMINATIM_*, GOOGLE_GEOCODING_API_KEY	nominatim	Geocoder choice; a Google key is needed only for google.
OSRM_BASE_URL, OSRM_PROFILE, OSRM_TIMEOUT_MS	empty, driving, 8000	Road routing. Empty means straight-line estimates. The public demo server is for development only.
OPTIMIZATION_PROVIDER	nn2opt	nn2opt, or mock for the old placeholder.
ROUTE_ALGORITHM	NN_2OPT	NN_2OPT or DBSCAN>NN_2OPT when a request does not choose.
ROUTE_DBSCAN_EPS_SECONDS, ROUTE_DBSCAN_MIN_POINTS	300, 2	DBSCAN neighbourhood radius (road seconds) and minimum cluster size.
ROUTE_2OPT_MAX_MS, ROUTE_2OPT_MAX_PASSES	1500, 500	Safety limits for 2-opt.
DEFAULT_SERVICE_TIME_MINUTES	3	Dwell time per stop when a delivery has none.

Postman app settings (PostmanApp/.env): API_BASE_URL (the backend's /api/v1 URL - use the PC's LAN IP on a physical device, 10.0.2.2 on the Android emulator) and MAP_STYLE_URL (a MapLibre-compatible OpenStreetMap

style; default OpenFreeMap Liberty, no key). The app's OSRM_BASE_URL setting is unused: routing is done by the backend.

9. Running and testing

Service	Port	Start
PostgreSQL + PostGIS	5432	Must be installed and running; create the database and enable PostGIS.
Backend API	4000	cd backend; npm install; npx prisma db push; npm run prisma:seed; npm run dev
Admin panel	5173	cd frontend; npm install; npm run dev
Postman app (web)	8081	cd PostmanApp; npm install --legacy-peer-deps; npx expo start --web (native builds need a development build: npx expo run:android / run:ios)

Seed accounts (development only, password ChangeMe123!): superadmin@postal.local (SUPER_ADMIN), admin.bhandup@postal.local (ADMIN), ramesh.kadam@postal.local and sunita.pawar@postal.local (POSTMAN). npm run prisma:seed:demo in backend adds 11 demo deliveries with real Bhandup West coordinates (and resets them to ASSIGNED when re-run).

Automated checks. Backend: npm test (vitest, 136 tests across 8 files: algorithms, DBSCAN, routing and caching, planner freshness, status state machine, event listener, HTTP API), npm run typecheck, npm run build. Postman app: npx jest (173 tests in 23 suites), npx tsc --noEmit, npx eslint . --ext .ts,.tsx (type checks and lint are clean).

10. Known limitations

- **No delivery weight or vehicle capacity** in the data model. Parcel count is used as the carried load; hard capacity limits and multi-trip routes would need Delivery.weightGrams, Vehicle.maxPayloadKg and a weight column in the import mapping. There are also no delivery time windows.
- **Cost weights** (load 0.35, priority 0.25) and the DBSCAN radius (300 s) are sensible defaults, not calibrated against real data.
- **OSRM** is not bundled: without OSRM_BASE_URL routes use flagged straight-line estimates; the public demo server is development-only and receives delivery coordinates. A single road-matrix request is limited to about 100 points; larger rounds fall back to estimates.
- **Notifications** are stored but never created by the backend, and there is no server-driven push to the mobile app.
- **Scanned PDFs** cannot be imported (no OCR); use CSV, XLSX or a text-based PDF.
- **Native map screen** (iOS / Android) compiles and is lint-clean but has not been run on a device or emulator in this environment; it has no direction arrows. The web build of the app was exercised end to end in a browser.
- **Admin panel** pages for vehicles, notifications and research analytics exist in the source but are not routed or linked; the Route / RouteStop tables are unused.
- **Stats inconsistency (existing):** the postman stats endpoint counts RECIPIENT_UNAVAILABLE as remaining and does not include CANCELLED in completed, failed or remaining, so the four numbers can add up to less than the total.
- **Backend lint** (npm run lint) cannot run because ESLint is not installed in the backend project.

This document was generated from the source code as it stands today. Where the system does not do something (weight, push notifications, OCR), it says so rather than implying it does.